

Homework 01

Question 1 (Least squares and matrix calculations)

Set the random seed to 43201. Generate $n = 100$ observations with $p = 5$ predictors according to

$$x_{ij} \stackrel{\text{iid}}{\sim} N(0, 1), \quad \varepsilon_i \stackrel{\text{iid}}{\sim} N(0, 0.7^2),$$

with all predictors and errors generated independently. Let

$$\mathbf{X} = [\mathbf{1} \quad \mathbf{x}_1 \quad \cdots \quad \mathbf{x}_5], \quad \beta = (3, 1.4, -0.9, 0.7, 0, 1.1)^\top,$$

and generate

$$\mathbf{y} = \mathbf{X}\beta + \varepsilon.$$

- Generate the data and report the dimensions and rank of $\mathbf{X}$. Calculate the least-squares estimate $\hat{\beta}$ using a numerically stable least-squares routine, without explicitly calculating $(\mathbf{X}^\top \mathbf{X})^{-1}$. Compare the estimates with the coefficients used to generate the data.
- Calculate the fitted values, the residual vector $\mathbf{r} = \mathbf{y} - \mathbf{X}\hat{\beta}$, and the training root mean squared error

$$\text{RMSE} = \left(\frac{1}{n} \sum_{i=1}^n r_i^2 \right)^{1/2}.$$

- Report $\|\mathbf{X}^\top \mathbf{r}\|_\infty$. Explain why this value should be close to zero and what it tells us about the residual vector.

Question 1 Answer

Part (a)

```
set.seed(43201)

n <- 100
p <- 5

# Generate the predictors and name the columns to match the notation.
x <- matrix(rnorm(n * p), nrow = n, ncol = p)
colnames(x) <- paste0("x", seq_len(p))

X <- cbind("(Intercept)" = 1, x)
beta <- c(
  "(Intercept)" = 3,
  x1 = 1.4,
  x2 = -0.9,
  x3 = 0.7,
  x4 = 0,
  x5 = 1.1
)

epsilon <- rnorm(n, mean = 0, sd = 0.7)
y <- as.vector(X %*% beta + epsilon)

# lm.fit uses a stable QR-based least-squares calculation, avoiding an
# explicit  $(X^T X)^{-1}$ .
fit <- lm.fit(x = X, y = y)
beta_hat <- fit$coefficients

coefficient_comparison <- data.frame(
  coefficient = names(beta),
  truth = unname(beta),
  estimate = unname(beta_hat)
)

print(dim(X))
```

```
[1] 100  6
```

```
print(qr(X)$rank)
```

```
[1] 6
```

```
print(coefficient_comparison)
```

	coefficient	truth	estimate
1	(Intercept)	3.0	3.0063210
2	x1	1.4	1.4171604
3	x2	-0.9	-0.8883453
4	x3	0.7	0.7952834
5	x4	0.0	0.0484378
6	x5	1.1	1.1319341

$\mathbf{X}$ has $n = 100$ rows and $p + 1 = 6$ columns (the intercept column plus the five predictors), and it has full column rank 6. Using `lm.fit()`, which relies on a QR decomposition rather than forming $(\mathbf{X}^\top \mathbf{X})^{-1}$ explicitly, gives

$$\hat{\beta} \approx (3.006, 1.417, -0.888, 0.795, 0.048, 1.132)^\top.$$

This is close to the generating coefficients $\beta = (3, 1.4, -0.9, 0.7, 0, 1.1)^\top$; in particular, the estimated coefficient of x_4 is near its true value of zero. The estimates are not exact because this is a single noisy realization of $\mathbf{y}$.

Part (b)

```
fitted_values <- as.vector(X %*% beta_hat)
r <- y - fitted_values
training_rmse <- sqrt(mean(r^2))

print(training_rmse)
```

```
[1] 0.6763696
```

The fitted values are $\hat{\mathbf{y}} = \mathbf{X}\hat{\beta}$ and the residuals are $\mathbf{r} = \mathbf{y} - \hat{\mathbf{y}}$. The training RMSE is

$$\text{RMSE} = \left(\frac{1}{n} \sum_{i=1}^n r_i^2 \right)^{1/2} \approx 0.676,$$

which is close to the error standard deviation 0.7 used to simulate the data, as expected.

Part (c)

```
normal_equation_error <- max(abs(crossprod(X, r)))
print(normal_equation_error)
```

```
[1] 3.019807e-14
```

$$\|\mathbf{X}^\top \mathbf{r}\|_\infty \approx 3.0 \times 10^{-14}.$$

This value should be (numerically) zero because $\hat{\beta}$ minimizes $\|\mathbf{y} - \mathbf{X}\beta\|_2^2$, and setting the gradient $\nabla L(\beta) = -2\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\beta)$ to zero at the minimizer gives the normal equations $\mathbf{X}^\top \mathbf{r} = \mathbf{0}$. The tiny nonzero value we see is just floating-point rounding error. Geometrically, this means the residual vector is orthogonal to every column of $\mathbf{X}$ (including the intercept column, so the residuals also sum to approximately zero) — the least-squares fit has extracted all of the signal that is linearly explainable by $\mathbf{X}$, leaving a residual that is uncorrelated with every predictor.

Question 2 (Gaussian likelihood for linear regression)

Continue with the data from Question 1. Suppose

$$\mathbf{Y} \mid \mathbf{X} \sim N_n(\mathbf{X}\beta, \sigma^2 \mathbf{I}_n).$$

The log-likelihood, including the constant term, is

$$\ell(\beta, \sigma^2) = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} (\mathbf{y} - \mathbf{X}\beta)^\top (\mathbf{y} - \mathbf{X}\beta).$$

- For fixed σ^2 , explain why maximizing $\ell(\beta, \sigma^2)$ over β is equivalent to minimizing the residual sum of squares. What does this imply about the maximum-likelihood estimate of β ?

- b. For fixed β , differentiate the log-likelihood with respect to σ^2 and derive its maximum-likelihood estimate. Calculate this estimate at $\hat{\beta}$ and compare it with the unbiased estimate of σ^2 . Explain why the two denominators differ.
- c. Let

$$\mathbf{e}_{x_2} = (0, 0, 1, 0, 0, 0)^\top.$$

Using the variance estimate from part b, plot

$$\ell(\hat{\beta} + t\mathbf{e}_{x_2}, \hat{\sigma}_{\text{MLE}}^2)$$

over a grid of t values from -1.25 to 1.25 . State where the maximum occurs and explain why this agrees with the least-squares result.

Question 2 Answer

Part (a)

For fixed σ^2 , the first term $-\frac{n}{2} \log(2\pi\sigma^2)$ does not depend on β , and the coefficient $-\frac{1}{2\sigma^2}$ multiplying the sum of squares is strictly negative. So maximizing $\ell(\beta, \sigma^2)$ over β is the same as maximizing a negative multiple of $\|\mathbf{y} - \mathbf{X}\beta\|_2^2$, which is the same as minimizing $\|\mathbf{y} - \mathbf{X}\beta\|_2^2$ itself:

$$\operatorname{argmax}_{\beta} \ell(\beta, \sigma^2) = \operatorname{argmin}_{\beta} \|\mathbf{y} - \mathbf{X}\beta\|_2^2.$$

This means the maximum-likelihood estimate of β is exactly the least-squares estimate $\hat{\beta}$ from Question 1, regardless of the (unknown) value of σ^2 .

Part (b)

Writing $v = \sigma^2$ and holding β fixed at $\text{RSS}(\beta) = \|\mathbf{y} - \mathbf{X}\beta\|_2^2$,

$$\ell(\beta, v) = -\frac{n}{2} \log(2\pi v) - \frac{\text{RSS}(\beta)}{2v}, \quad \frac{\partial \ell}{\partial v} = -\frac{n}{2v} + \frac{\text{RSS}(\beta)}{2v^2}.$$

Setting this to zero and solving gives

$$\hat{\sigma}_{\text{MLE}}^2 = \frac{\text{RSS}}{n},$$

and the second derivative $-n/(2\hat{v}^2) < 0$ confirms it is a maximum.

```
rss <- sum(r^2)
sigma2_mle <- rss / n
sigma2_unbiased <- rss / (n - ncol(X))

print(c(sigma2_mle = sigma2_mle, sigma2_unbiased = sigma2_unbiased))
```

```
sigma2_mle sigma2_unbiased
0.4574759      0.4866765
```

At $\hat{\beta}$, $\hat{\sigma}_{\text{MLE}}^2 \approx 0.4575$, compared with the unbiased estimate $\hat{\sigma}_{\text{unbiased}}^2 = \text{RSS} / (n - 6) \approx 0.4867$. The denominators differ because the MLE divides by n (it directly maximizes the likelihood), while the unbiased estimator divides by $n - 6$ to account for the six regression coefficients estimated from the data — under the Gaussian linear model, $\mathbb{E}(\text{RSS}) = (n - 6)\sigma^2$, so dividing by n instead of $n - 6$ makes $\hat{\sigma}_{\text{MLE}}^2$ slightly biased downward.

Part (c)

```
log_likelihood <- function(beta_value, sigma2, X, y) {
  residual <- y - as.vector(X %*% beta_value)
  -nrow(X) / 2 * log(2 * pi * sigma2) - sum(residual^2) / (2 * sigma2)
}

e_x2 <- c(0, 0, 1, 0, 0, 0)
t_grid <- seq(-1.25, 1.25, length.out = 251)
profile_log_likelihood <- vapply(
  t_grid,
  function(t) log_likelihood(beta_hat + t * e_x2, sigma2_mle, X, y),
  numeric(1)
)

print(c(maximizing_t = t_grid[which.max(profile_log_likelihood)]))
```

```
maximizing_t
0
```

```
plot(
  t_grid, profile_log_likelihood, type = "l", lwd = 2, col = "#1F5A94",
  xlab = expression(t), ylab = "Log-likelihood"
)
abline(v = 0, lty = 2, lwd = 2, col = "#C84A16")
```

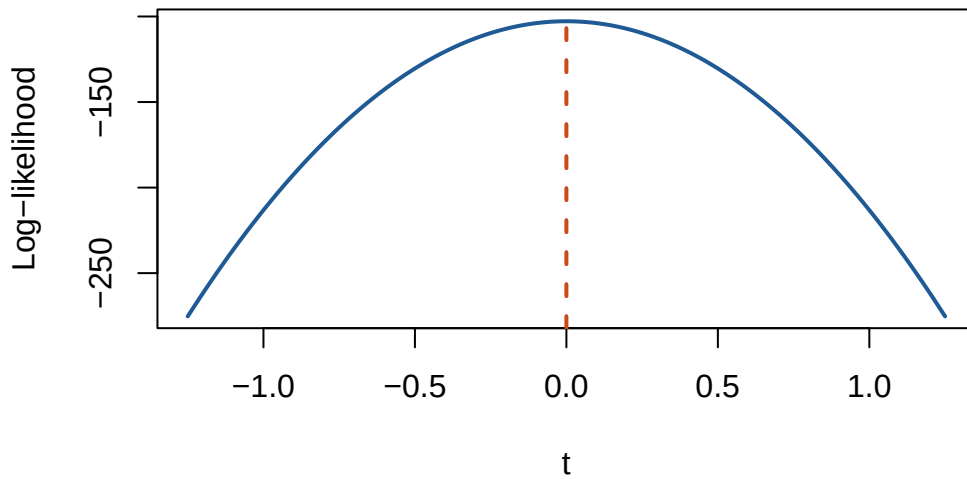


Figure 1: Likelihood profile for the coefficient of x_2 .

The maximum occurs at $t = 0$, i.e., at $\hat{\beta}$ itself. This agrees with the least-squares result because part (a) showed that the MLE of β (for any fixed σ^2 , including $\hat{\sigma}_{\text{MLE}}^2$) is the least-squares estimate. Moving away from $t = 0$ only changes the coefficient of x_2 , which increases the residual sum of squares and therefore decreases the log-likelihood.

Question 3 (Starting values in nonconvex optimization)

Consider the function

$$f(x) = \exp(1.5x) - 3(x + 6)^2 - 0.05x^3,$$

with derivative

$$f'(x) = 1.5 \exp(1.5x) - 6(x + 6) - 0.15x^2.$$

- Plot $f(x)$ over the interval $[-40, 7]$. Based on the plot, describe the important features of the objective function that may affect numerical optimization.
- Use BFGS to minimize $f(x)$ twice, starting at $x^{(0)} = -15$ and $x^{(0)} = 0$. Supply $f'(x)$ to the optimizer. For each run, report the final value of x , the final objective value, $|f'(x)|$, and whether the optimizer reported convergence.
- Explain why the two runs can converge to different answers. Which run gives the lower objective value? What additional evidence would be needed before claiming that this point is the global minimum?

Question 3 Answer

Part (a)

```
objective <- function(x) exp(1.5 * x) - 3 * (x + 6)^2 - 0.05 * x^3
gradient <- function(x) 1.5 * exp(1.5 * x) - 6 * (x + 6) - 0.15 * x^2

x_grid <- seq(-40, 7, length.out = 1200)

# Restrict only the displayed vertical range so both minima are visible;
# this changes the plot, not the function seen by the optimizer.
plot(
  x_grid, objective(x_grid), type = "l", lwd = 2, col = "#13294B",
  ylim = c(-450, 100), xlab = "x", ylab = "f(x)"
)
```

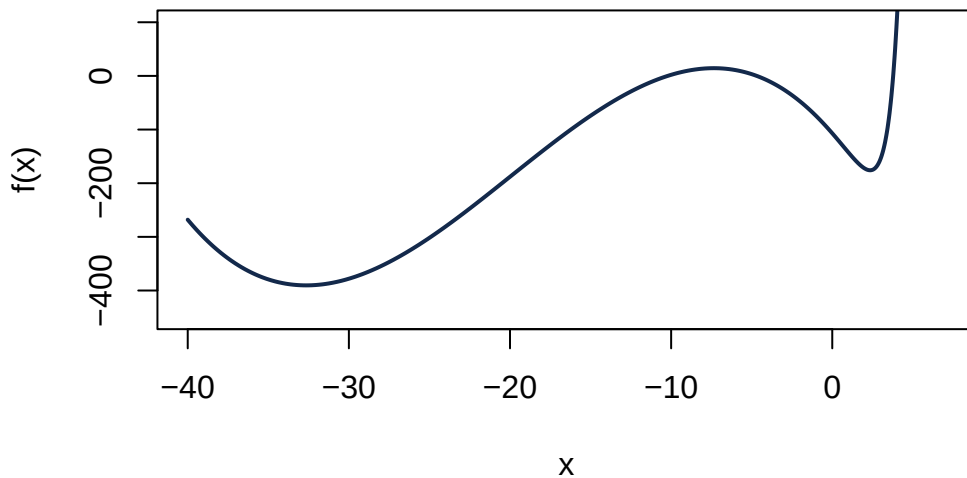


Figure 2: Objective function over $[-40, 7]$.

The plot shows two local minima separated by a local maximum, and the function rises very sharply as x approaches the right edge of the interval (driven by the $\exp(1.5x)$ term). Because there are multiple local minima, a gradient-based optimizer's result will depend on which "basin" its starting point falls into, and the steep region on the right could also cause numerical overflow or very large gradients if a starting point or step lands there.

Part (b)

```
starts <- c(-15, 0)
optimization_results <- lapply(
  starts,
  function(start) optim(par = start, fn = objective, gr = gradient, method = "BFGS")
)

optimization_summary <- do.call(rbind, Map(
  function(start, result) {
    data.frame(
      starting_value = start,
      final_x = result$par,

```

```

        objective = result$value,
        absolute_gradient = abs(gradient(result$par)),
        converged = result$convergence == 0
    )
},
starts, optimization_results
))

print(optimization_summary)

```

	starting_value	final_x	objective	absolute_gradient	converged
1	-15	-32.649111	-390.3858	8.725087e-08	TRUE
2	0	2.349967	-175.8626	6.556057e-07	TRUE

Starting from $x^{(0)} = -15$, BFGS converges to approximately $x \approx -32.649$ with $f(x) \approx -390.386$. Starting from $x^{(0)} = 0$, it converges to approximately $x \approx 2.350$ with $f(x) \approx -175.863$. Both runs report a gradient near zero and convergence.

Part (c)

The two runs converge to different answers because f is nonconvex with two separate local minima, and BFGS only uses local gradient information — it moves downhill from its starting point and stops at whichever local minimum is in that basin of attraction, without “seeing” the other one.

The run starting at $x^{(0)} = -15$ gives the lower objective value (-390.386 versus -175.863). A near-zero gradient and a reported convergence flag only confirm that a point is a *local* stationary point — they say nothing about other regions of the domain. To claim the point found is the *global* minimum, we would want additional evidence such as: checking the behavior of $f(x)$ as $x \rightarrow -\infty$ and $x \rightarrow +\infty$, finding all roots of $f'(x)$ analytically or numerically and comparing $f(x)$ at each one, or running BFGS from many starting values across the domain and confirming none finds a lower value.

Question 4 (Nearly collinear predictors)

Continue with $\mathbf{X}$ and $\mathbf{y}$ from Question 1. Set the random seed to 43202 and generate

$$u_i \stackrel{\text{iid}}{\sim} N(0, 1), \quad i = 1, \dots, n.$$

Define

$$\mathbf{x}_6 = \mathbf{x}_1 + 10^{-4}\mathbf{u}, \quad \mathbf{X}_+ = [\mathbf{X} \quad \mathbf{x}_6],$$

and let

$$\mathbf{y}^* = \mathbf{y} + 10^{-3}\mathbf{u}.$$

Use a numerically stable least-squares routine throughout.

- Fit $\mathbf{y}$ using $\mathbf{X}$ and $\mathbf{X}_+$. For each design matrix, report the condition number and training RMSE. For the augmented fit, also report the coefficients of $\mathbf{x}_1$ and $\mathbf{x}_6$.
- Fit $\mathbf{y}^*$ using $\mathbf{X}_+$. Report the changes in the coefficients of $\mathbf{x}_1$ and $\mathbf{x}_6$. Compare the two fitted-value vectors using their root mean squared difference and maximum absolute difference.
- Use the identity

$$\mathbf{y}^* - \mathbf{y} = 10(\mathbf{x}_6 - \mathbf{x}_1)$$

to explain why the fitted values change very little while the two coefficients change substantially. What does this example suggest about interpreting separate effects for nearly collinear predictors?

Question 4 Answer

Part (a)

```
set.seed(43202)
u <- rnorm(n)

x6 <- x[, "x1"] + 1e-4 * u
X_plus <- cbind(X, x6 = x6)

fit_original <- lm.fit(x = X, y = y)
fit_augmented <- lm.fit(x = X_plus, y = y)

condition_number <- function(design) {
  singular_values <- svd(design, nu = 0, nv = 0)$d
  max(singular_values) / min(singular_values)
}
```

```

fit_summary <- data.frame(
  design = c("X", "X_plus"),
  condition_number = c(condition_number(X), condition_number(X_plus)),
  training_rmse = c(
    sqrt(mean(fit_original$residuals^2)),
    sqrt(mean(fit_augmented$residuals^2))
  )
)
print(fit_summary)

```

```

      design condition_number training_rmse
1        X           1.524914      0.6763696
2 X_plus      20209.203501      0.6754668

```

```

print(fit_augmented$coefficients[c("x1", "x6")])

```

```

      x1      x6
351.6766 -350.2588

```

The condition number jumps from about 1.5 for $\mathbf{X}$ to about 2.0×10^4 for $\mathbf{X}_+$, while the training RMSE barely changes (about 0.676 versus 0.675). In the augmented fit, the individual coefficients of $\mathbf{x}_1$ and $\mathbf{x}_6$ are large with opposite signs (approximately 351.68 and -350.26), even though $\mathbf{x}_6$ is almost identical to $\mathbf{x}_1$.

Part (b)

```

y_star <- y + 1e-3 * u
fit_perturbed <- lm.fit(x = X_plus, y = y_star)

coefficient_change <- fit_perturbed$coefficients[c("x1", "x6")] -
  fit_augmented$coefficients[c("x1", "x6")]

fitted_augmented <- as.vector(X_plus %*% fit_augmented$coefficients)
fitted_perturbed <- as.vector(X_plus %*% fit_perturbed$coefficients)
fitted_difference <- fitted_perturbed - fitted_augmented

print(coefficient_change)

```

```
x1  x6
-10 10
```

```
print(c(
  fitted_difference_rmse = sqrt(mean(fitted_difference^2)),
  fitted_difference_max = max(abs(fitted_difference))
))
```

```
fitted_difference_rmse  fitted_difference_max
0.001015879             0.002572675
```

Refitting with $\mathbf{y}^*$ instead of $\mathbf{y}$ changes $\hat{\beta}_1$ by about -10 and $\hat{\beta}_6$ by about $+10$ — large, exactly-opposite shifts. Yet the two fitted-value vectors are nearly identical: the root mean squared difference is about 0.00102 and the maximum absolute difference is about 0.00257.

Part (c)

Since $\mathbf{x}_6 - \mathbf{x}_1 = 10^{-4}\mathbf{u}$, the response perturbation satisfies

$$\mathbf{y}^* - \mathbf{y} = 10^{-3}\mathbf{u} = 10^{-3} \cdot \frac{\mathbf{x}_6 - \mathbf{x}_1}{10^{-4}} = 10(\mathbf{x}_6 - \mathbf{x}_1).$$

So the augmented model can reproduce this exact change in $\mathbf{y}$ purely by shifting $\hat{\beta}_1 \rightarrow \hat{\beta}_1 - 10$ and $\hat{\beta}_6 \rightarrow \hat{\beta}_6 + 10$ (leaving other coefficients essentially unchanged), because $-10\mathbf{x}_1 + 10\mathbf{x}_6 = 10(\mathbf{x}_6 - \mathbf{x}_1) = \mathbf{y}^* - \mathbf{y}$ exactly matches the change needed in fitted values. This is a huge change in the individual coefficients but corresponds to a tiny change in predictions, since $\mathbf{x}_1$ and $\mathbf{x}_6$ are nearly collinear: the data can pin down $\hat{\beta}_1 + \hat{\beta}_6$ (their combined contribution) quite precisely, but cannot separately identify $\hat{\beta}_1$ and $\hat{\beta}_6$ individually — small amounts of noise can swing them wildly in opposite directions. This means we should not trust or interpret the individual coefficients of nearly collinear predictors as separate effects; only their combined effect is well identified by the data.

Question 5 (Data preparation and summary statistics)

The file `data/data-manipulation.csv` is a small constructed data table containing an observation identifier, a group label, two numeric predictors, and a response. Some response values are missing. For analyses involving the response, use only observations with a recorded response. Do not replace a missing response with zero or a group mean.

- a. Read the data and report its dimensions, column types, number of duplicated identifiers, and number of missing values in each column. Create an analysis table containing only observations with a recorded response, and define

```
feature_sum = x1 + x2.
```

- b. For each group, report the number of observations, the mean response, and the mean of `feature_sum`. State clearly which observations these summaries describe.
- c. Sort the analysis table by response from largest to smallest and report the first three rows, including `observation_id`, `group`, `response`, and `feature_sum`. Add code checks verifying that the number of retained rows equals the number of nonmissing responses in the original data, that the analysis table has no missing response values, and that its identifiers are unique.

Question 5 Answer

Part (a)

```
data_file <- file.path("data", "data-manipulation.csv")
small_data <- read.csv(data_file, stringsAsFactors = FALSE, check.names = FALSE)

print(dim(small_data))
```

```
[1] 24  5
```

```
print(vapply(small_data, class, character(1)))
```

observation_id	group	x1	x2	response
"character"	"character"	"numeric"	"numeric"	"numeric"

```
print(c(duplicated_identifiers = sum(duplicated(small_data$observation_id))))
```

```
duplicated_identifiers
0
```

```
print(colSums(is.na(small_data)))
```

observation_id	group	x1	x2	response
0	0	0	0	3

```
# Keep only recorded responses for response-based summaries.
analysis_data <- small_data[!is.na(small_data$response), ]
analysis_data$feature_sum <- analysis_data$x1 + analysis_data$x2
```

The table has 24 rows and 5 columns. `observation_id` and `group` are character columns, while `x1`, `x2`, and `response` are numeric. There are no duplicated identifiers, and only `response` has missing values (3 of them). The analysis table (dropping the 3 missing-response rows) has 21 observations, with `feature_sum = x1 + x2` added.

Part (b)

```
group_summary <- do.call(rbind, lapply(
  split(analysis_data, analysis_data$group),
  function(group_data) {
    data.frame(
      group = group_data$group[1],
      observations = nrow(group_data),
      mean_response = mean(group_data$response),
      mean_feature_sum = mean(group_data$feature_sum),
      row.names = NULL
    )
  }
))
print(group_summary)
```

	group	observations	mean_response	mean_feature_sum
A	A	7	5.600000	3.600000
B	B	7	7.602857	4.928571
C	C	7	7.997143	6.585714

These summaries (7 observations per group, mean response and mean `feature_sum` shown above) describe only the 21 observations that have a recorded response — they should not be interpreted as summaries of all 24 rows in the original file.

Part (c)

```
largest_three <- analysis_data[
  order(analysis_data$response, decreasing = TRUE),
  c("observation_id", "group", "response", "feature_sum")
][1:3, ]

number_recorded <- sum(!is.na(small_data$response))
stopifnot(
  nrow(analysis_data) == number_recorded,
  !anyNA(analysis_data$response),
  !anyDuplicated(analysis_data$observation_id)
)

print(largest_three)
```

	observation_id	group	response	feature_sum
22	obs-22	C	9.24	6.6
16	obs-16	B	9.00	5.2
14	obs-14	B	8.66	5.0

The three largest recorded responses are **obs-22** (group C, response 9.24), **obs-16** (group B, response 9.00), and **obs-14** (group B, response 8.66). The `stopifnot()` checks confirm the analysis table's row count matches the number of nonmissing responses in the original data, that it has no missing responses, and that its identifiers are unique — all checks pass silently. Replacing a missing response with zero (or a group mean) would introduce values that were never observed and would distort the group summaries above.

Question 6 (Can 39 million Fitbit records represent US adults?)

[Patten et al. \(2026\)](#) describe Fitbit data from 59,018 participants in the All of Us Research Program. The dataset spans 14 years and contains more than 39 million daily step records. Participants contributed data through one of two routes:

- Bring Your Own Device (BYOD): participants shared data from a Fitbit they already owned.
- Wearables Enhancing All of Us Research (WEAR): invited participants received a Fitbit at no cost.

In the general activity cohort, the BYOD and WEAR groups contained 32,035 and 22,474 participants, respectively. The BYOD value is listed first in each comparison below:

- 77.3% versus 55.1% reported being White;
- 6.1% versus 15.2% reported annual household income between \$10,000 and \$25,000; and
- median daily steps were 6,867 versus 5,797.

Suppose the target is the mean of the participant-specific average daily step counts among US adults during the study period, with each adult given equal weight. A researcher writes:

“This dataset contains more than 39 million daily Fitbit records. Therefore, the average of all recorded step counts should provide an accurate estimate of mean daily activity among US adults.”

- Are the 39 million daily records independent observations? What characteristics or behaviors could cause some participants to contribute more recorded days than others? Explain how averaging all recorded days would then weight participants unequally.
- BYOD participants reported a median of 6,867 daily steps, compared with 5,797 among WEAR participants. Does this comparison show that already owning a Fitbit causes people to walk more? Explain your answer and give at least one plausible alternative explanation based on how participants entered the two groups.
- A very large dataset can still produce a biased estimate of a population quantity when the people and observations entering the dataset are selected. Using this study, explain how selection of both participants and recorded days could make the observed data differ from the US adult population. Why might the naive average of all recorded daily step counts fail to estimate the stated target? You do not need to determine the direction of the bias.

Source

Patten, T., Preble, E. A., Master, H., et al. (2026). [The All of Us Research Program’s wearables dataset](#). *Nature Medicine*, 32, 2302-2310.

Question 6 Answer

Part (a)

The 39 million daily records are not independent observations — they are repeated measurements taken on roughly 59,000 participants, and days from the same participant tend to be correlated (through that person’s health, occupation, habits, and environment, which persist over time). Participants can also differ systematically in how many days they contribute: longer enrollment, more consistent device use, regular syncing, and fewer technical problems or invalid days all increase a participant’s record count, while illness, travel, irregular schedules, privacy concerns, or device failure can reduce it. If we simply average all 39 million daily records together,

participants who happen to contribute more days get more influence on the average than participants who contribute fewer days — the average is implicitly weighted by “days recorded,” not by “one vote per adult,” which is not the equal-weighting target described in the problem.

Part (b)

No, this comparison does not show that already owning a Fitbit *causes* higher step counts. BYOD and WEAR are not randomly assigned groups — people self-select into BYOD by already owning a compatible device, whereas WEAR participants are invited and given a free device. The reported demographic differences (race, income) between the two groups are direct evidence that the two groups differ in ways beyond device ownership. A plausible alternative explanation: people who already own and use a Fitbit may be systematically more health-conscious, wealthier, or more physically active to begin with — the ownership itself may be a marker of preexisting higher activity levels and resources, not a cause of them. So the step-count gap could simply reflect who chose (or was able) to already own a device, rather than any effect of ownership on behavior.

Part (c)

A US adult has to pass through several selective stages before contributing any observed step count: (1) voluntarily enrolling in All of Us, which already may skew toward people more engaged with health research or health care; (2) entering via BYOD (requires already owning a compatible device) or WEAR (requires being invited and agreeing to use one); (3) consenting to share wearable data and successfully connecting an account; and (4) continuing to wear and sync the device so that valid daily records are produced. Selection can occur at every one of these stages, and if any of it relates to physical activity (which seems likely, since device ownership and consistent use both plausibly correlate with health behavior), the resulting sample of participants — and the sample of days they contribute — need not look like the general US adult population, no matter how large the raw record count is. Combined with the unequal-weighting problem from part (a), the naive average of all 39 million records conflates two separate issues: it doesn’t give each adult equal weight, and even a properly participant-weighted average would still describe only the selected participants who made it through enrollment, device access, consent, and continued use — not necessarily the broader US adult population. A large sample size only reduces *random* sampling variability; it does nothing to correct a *systematic* selection bias like this.